SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: ***Total Marks: 50***

Annexure A

Debugging & Optimization Task

Code of Conduct:

I ***Sarika K S 192521039*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

Debugging & Optimisation Task

QUESTION 3:
DEBUGGING RSA CRYPTOSYSTEM IMPLEMENTATION

Problem statement:

A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.

1. Identification of Errors and Root Causes

- **Prime generation errors:** Using small, predictable, repeated, or non-prime values for p and q; failing to check primality properly; or accidentally choosing p = q. RSA requires two distinct large primes.
- **Incorrect key generation:** Calculating n incorrectly instead of n = p × q; using φ(n) = (p − 1)(q − 1) incorrectly; choosing e where gcd(e, φ(n)) ≠ 1; or computing d incorrectly. The private exponent must satisfy (e × d) mod φ(n) = 1 (or use the corresponding Carmichael function).
- **Incorrect modular exponentiation:** Using ordinary exponentiation such as pow(m, e) followed by a modulus can create unnecessarily large intermediate values. The correct operation is modular exponentiation: c = m^e mod n and m = c^d mod n.
- **Large-message handling:** RSA can only process an integer smaller than n. Encrypting an entire long message as one integer may exceed n and produce errors or irreversible truncation. RSA is not designed to encrypt large files directly.
- **Encoding and block errors:** Converting text to an integer and back inconsistently, losing leading zero bytes, using incorrect block sizes, or mixing byte order can result in plaintext that does not match the original.
- **Padding and security errors:** Raw textbook RSA is deterministic and insecure for real applications. Incorrect or missing padding can also cause decryption failures and expose the system to attacks.

B. Optimization Using Fast Exponentiation and the Chinese Remainder Theorem

2. Correct RSA Operations

RSA key generation follows these steps:

1. Choose two distinct large primes p and q.
2. Compute n = p × q and φ(n) = (p − 1)(q − 1).
3. Choose a public exponent e such that 1 < e < φ(n) and gcd(e, φ(n)) = 1 (65537 is commonly used).
4. Compute the private exponent d as the modular inverse of e modulo φ(n), so (e × d) mod φ(n) = 1.

5. The public key is (n, e); the private key contains d and the prime factors p and q.

3. Efficient Modular Exponentiation

Use Python's three-argument pow(base, exponent, modulus). It performs modular exponentiation efficiently without constructing the full power first.

Encryption: c = pow(m, e, n)

Decryption: m = pow(c, d, n)

Before encryption, verify $0 \leq m < n$. For long messages, use a standard hybrid-encryption design: encrypt the data with an authenticated symmetric cipher and encrypt only the randomly generated symmetric key with RSA-OAEP. Do not split arbitrary data into raw RSA blocks.

4. Example Python Demonstration

```
from math import gcd

def make_rsa_keys(p, q, e=65537):
    # Demonstration only: p and q must be distinct primes.
    if p == q or p < 2 or q < 2:
        raise ValueError("p and q must be distinct primes")
    n = p * q
    phi = (p - 1) * (q - 1)
    if gcd(e, phi) != 1:
        raise ValueError("e must be relatively prime to phi(n)")
    d = pow(e, -1, phi)  # modular inverse
    return (n, e), (n, d)

def rsa_encrypt_integer(message_int, public_key):
    n, e = public_key
    if not 0 <= message_int < n:
        raise ValueError("Message integer must be in the range [0, n)")
    return pow(message_int, e, n)

def rsa_decrypt_integer(ciphertext_int, private_key):
    n, d = private_key
    if not 0 <= ciphertext_int < n:
        raise ValueError("Ciphertext integer must be in the range [0, n)")
    return pow(ciphertext_int, d, n)

# Small primes are used only to demonstrate correctness, not security.
public_key, private_key = make_rsa_keys(61, 53, e=17)
message = 65
ciphertext = rsa_encrypt_integer(message, public_key)
recovered = rsa_decrypt_integer(ciphertext, private_key)
assert recovered == message
print("Ciphertext:", ciphertext)
print("Recovered message:", recovered)
```

Note: This example uses tiny primes and raw RSA only to illustrate arithmetic. It is not secure and must not be used for real messages. In production, use a vetted cryptographic library with RSA-OAEP and properly generated keys.

5. Chinese Remainder Theorem (CRT) Optimization

CRT speeds up RSA private-key operations by performing exponentiation modulo p and q separately, then recombining the results. Precompute dp = d mod (p − 1), dq = d mod (q − 1), and qInv = q^{-1} mod p.

- m1 = c^dp mod p
- m2 = c^dq mod q
- h = qInv × (m1 − m2) mod p
- m = m2 + h × q

In Python, each power can use pow(c, dp, p) or pow(c, dq, q). CRT commonly makes private-key operations several times faster, although implementations must protect against fault and side-channel attacks.

C. Trade-offs Between Key Size, Security, and Performance

Factor	Effect	Recommendation
Key size	Larger RSA moduli increase the difficulty of factoring but make key generation and exponentiation slower and increase key/ciphertext size.	Use a currently accepted key size based on organizational policy; 2048-bit RSA is a common minimum baseline, while higher assurance may require 3072 bits or more.
Public exponent	A small, well-chosen public exponent such as 65537 allows efficient public-key operations.	Use a standard exponent and a trusted library.
Private operations	Private exponentiation is computationally expensive; CRT reduces its cost.	Use CRT through a vetted library and keep private-key material protected.
Message size	RSA can encrypt only a short payload relative to the modulus and padding overhead.	Use hybrid encryption: RSA-OAEP for a symmetric key, authenticated symmetric encryption for data.
Implementation risk	Custom prime generation, padding, encoding, and CRT code can introduce serious flaws.	Prefer maintained cryptographic libraries, secure randomness, standard padding, and test vectors.

6. Testing and Validation

- Check that p and q are distinct primes and that n = p × q.
- Check gcd(e, φ(n)) = 1 and (e × d) mod φ(n) = 1.
- Test round-trip encryption/decryption for valid integers smaller than n.
- Test boundary values such as 0, 1, and n − 1, plus invalid values outside the range.
- Test byte encoding, leading zero bytes, padding errors, and messages larger than the permitted RSA payload.
- Benchmark key generation, encryption, and decryption separately using realistic key sizes and message workloads.

Conclusion

The RSA failures are most likely caused by incorrect prime/key generation, invalid message sizes, inefficient or incorrect modular exponentiation, or inconsistent message encoding. Correct key mathematics and validation prevent many logic errors. Python's modular pow() improves exponentiation efficiency, while CRT accelerates private-key operations. Larger keys provide stronger protection but cost more computation. For real systems, use a vetted cryptographic library and RSA-OAEP in a hybrid-encryption design rather than implementing raw RSA yourself.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation